

**Road Safety Problems Caused by Poor Street Lighting in Tala, Zone 2,
Lumbangan, Barangay Boalan, Zamboanga City**

INTRODUCTION

The basic duty of local government authorities consists of protecting their residents who depend on governmental entities for their safety. The city of Zamboanga now faces greater challenges in public safety maintenance because its urban areas have expanded and its residential areas have become more populated. The community of Tala Zone 2 Lumbangan in Barangay Boalan faces its most serious threat because the area lacks proper infrastructure that includes operational street lights which keeps all residents safe.

The lack of dependable street lighting systems stands as the main reason why road accidents and nighttime crashes have risen throughout the region. People who drive and walk on local streets face navigation challenges because they cannot see hidden dangers which include potholes and sharp curves and crossing pedestrians during dark times. The system limits visibility which people need for safe movement because it creates unsafe conditions that lead to vehicle accidents and make streets dangerous after dark. The World Health Organization (2018) reports that urban areas in developing countries experience high rates of road traffic injuries because their road systems lack proper infrastructure and sufficient street lighting.

The existing safety protocols which depend on driver alertness and standard road signs do not provide adequate protection during times of restricted visibility. The measures require sufficient illumination for their successful operation. The absence of light in Zone 2 creates "blind zones" which prevent people from seeing dangers and increase their risk of encountering accidents. The situation creates two problems for residents because they experience higher chances of physical harm and their anxiety about safety prevents them from going outside after dark.

According to the International Commission on Illumination (2010) proper lighting serves as a critical component which helps both drivers and pedestrians to reduce visual fatigue and enhance their ability to detect dangers. The path leads to the station. The station operates for 24 hours. The main entrance serves as the primary entry point. The building functions as an official government facility. The facility operates throughout the entire day and night. The system uses machine learning algorithms to process and analyze data. The institution provides its students with access to advanced educational facilities. The university offers

specialized programs to meet the needs of various professional fields. The website uses HTML markup to structure its content.

Empirical studies show that proper street lighting significantly improves road safety and public order. According to Elvik (2009) systems that have effective design can decrease nighttime traffic accidents up to 30% by providing better visibility which helps drivers to react quickly. The modern lighting technologies of LED and solar-powered systems provide sustainable and efficient solutions through their capacity to produce constant light while requiring less energy and maintenance work. The research investigates how street lighting deficiencies affect Tala Zone 2 Barangay Boalan while assessing how improved lighting solutions will lead to better visibility and lower accident rates which will create safer pathways for pedestrians thus building a more secure and accessible community.

OBJECTIVES OF THE PROJECT

This study generally aims to address the safety risks caused by insufficient street lighting in Tala, Zone 2, Barangay Boalan, Zamboanga City by proposing effective lighting and security improvements that can enhance visibility, reduce crime, and improve overall community safety, as proper street lighting systems are essential in improving visibility and safety in public areas (Welsh & Farrington, 2008; Ibrahim et al., 2020). More specifically, it aims to assess the current condition of street lighting in terms of brightness, coverage, and functionality; to identify areas with inadequate or non-functional lighting; to examine the effects of poor lighting on road safety and crime occurrence; to propose the use of energy-efficient lighting systems such as LED lights; to recommend the integration of street lighting with CCTV systems for improved security; and to evaluate how improved lighting can enhance the safety and well-being of residents.

PROPOSED SOLUTION

The proposed solution aims to prioritize public safety and sustainability through the installation of solar streetlights with the help of the renewable resource which is the sunlight to power the solar panels for our street lights. These lights are high-quality LED technology to ensure the streets are brighter and have more efficient illumination, reducing energy consumption and environmental impact and focus on natural resources. The placement of the lights will concentrate on high-risk areas to enhance safety for the community and commuters.

To optimize effectiveness and convenience, the system will include smart technology for automatic control that enables features such as dimming and automatic on/off functions. Assure regular maintenance and immediate repairs will be prioritized to ensure the lifespan and reliability of the lighting system. According to Sepco (2016), solar street lights have a 15-20 years lifespan and may reduce electricity use by up to 85%. (C40 , 2011). Additionally, the project will have a partnership between the government and private sector entities to guarantee support, funding, and technical expertise, fostering a sustainable and community-oriented approach to urban lighting.

Overall, the proposal of solar streetlights presents a sustainable, efficient, and reliable solution for urban illumination. By utilizing renewable sunlight and the use of advanced LED and smart technology, the project seeks to improve community safety while lessening environmental impact. Strategic placement in high-risk areas, paired with ongoing maintenance and collaborative effort from government and private sectors, will ensure the system's longevity and effectiveness. Ultimately, this initiative not only promotes sustainability and energy savings but also fosters a safer and more resilient urban environment.

BENEFITS OF THE PROBLEM

Proper street lighting provides many important benefits to the community, especially in areas such as Tala, Zone 2, Barangay Boalan, where poor lighting creates serious road safety concerns. One of the main benefits of adequate street lighting is improved visibility during nighttime. When roads, streets, and narrow pathways are well illuminated, drivers,

pedestrians, and cyclists can move more safely and confidently. Clear visibility helps prevent accidents caused by unseen obstacles, uneven roads, or approaching vehicles, which significantly improves overall road safety.

Another major benefit is the reduction of crime and the improvement of public security. According to Welsh et al. (2008), improved street lighting significantly reduces crime because better illumination increases visibility and surveillance, making it harder for offenders to act without being noticed. Their systematic review concluded that improved street lighting has clear benefits for law-abiding citizens and should continue to be used to prevent crime in public areas .

Street lighting also helps residents feel safer and more confident when walking or traveling at night. Dark streets often create fear and discomfort, especially for students, workers, and families returning home late. With reliable lighting systems, people can move freely and perform daily activities without unnecessary fear. This also encourages stronger community interaction and supports local economic activity during evening hours.

In addition, using modern lighting technologies such as LED streetlights offers long-term economic and environmental advantages. LED lighting consumes less electricity, lasts longer, and requires less maintenance compared to traditional lighting systems. This makes it a cost-effective and sustainable solution for local government units and communities aiming to improve public infrastructure.

Overall, proper street lighting is essential for promoting road safety, reducing accidents, preventing crime, and improving the quality of life of residents. It creates a safer, more secure, and more comfortable environment for everyone while supporting community development and public well-being.

SCOPE AND DELIMITATION

- **Geographic Focus (Location)** The physical boundaries of this project are strictly delimited to the primary roads, interior streets, and identified "Dark Areas" within Tala, Zone 2, Barangay Boalan, Zamboanga City. The study focuses on these specific high-risk corridors where the absence of functional lighting infrastructure has been identified as a primary factor in local safety concerns.
- **Operational Scope (Action)** The project encompasses the end-to-end technical process of improving public visibility. This includes conducting a comprehensive site and soil survey, the procurement of high-durability LED and solar-powered lighting systems, and the physical installation of the hardware. The scope also includes the strategic placement of poles based on "luminous flux" requirements to ensure maximum light coverage.
- **Target Beneficiaries (Target)** The project outcomes are specifically designed to benefit the active population of Zone 2, including residents, nighttime commuters, and motorists. The scope of safety assessment is limited to the reduction of road accidents and the deterrence of criminal activities facilitated by low-visibility conditions within the specified area.
- **Strategic Engagement (Liaison)** The proposal includes the management of institutional and community relations. This involves formal coordination with the Local Government Unit (LGU) and the Department of Public Works for permitting and right-of-way clearances. Furthermore, the scope includes a digital transparency initiative to provide project updates to the community via social media platforms.
- **Project Timeline (Duration)** The execution of the project is delimited to a fixed eight-month duration. This timeframe accounts for the preliminary phase (site assessment and permitting), the middle phase (procurement and logistics), and the final phase (physical installation, testing, and project turnover). Any maintenance or monitoring beyond this eight-month window is considered outside the initial implementation phase.
- **Project Boundaries (Exclusions)** To maintain fiscal and operational focus, the project excludes any electrical repairs or installations within private residential properties. Additionally, it does not cover lighting infrastructure in neighboring zones or the broader city-wide electrical grid maintenance. Long-term electricity costs and

major grid upgrades are the responsibility of the local electric cooperative or the municipal government and are not included in this proposal.

Implementation Plan

This study aims to enhance street lighting conditions in Tala, Zone 2, Barangay Boalan, Zamboanga City by improving nighttime visibility, reducing crime rates, and strengthening overall community safety. As a vital component of urban infrastructure, effective street lighting promotes safer public spaces, supports mobility, and fosters a greater sense of security among residents.

- **Planning and Approval**
 - This phase establishes clear objectives, secures necessary approvals, and prepares the resources needed to ensure the project starts efficiently and with proper coordination.
- **Phase 2: Site Assessment**
 - This phase evaluates current lighting conditions by identifying dark areas, safety risks, and gathering community feedback to address real needs effectively.
- **Phase 3: Design and Installation**
 - This phase focuses on creating an efficient lighting plan, acquiring quality materials, and installing streetlights to improve visibility and safety.
- **Phase 4: Evaluation and Maintenance**
 - This phase ensures long-term success by monitoring performance, maintaining lighting systems, and continuously improving safety and reliability.

Time Table

Project Scoping & Compliance	May 20, 2026 – June 25, 2026	Establish research objectives, define the problem statement, and conduct coordination meetings with stakeholders and barangay officials for formal approval.
Site Audit & Baseline Study	June 26, 2026 – August 10, 2026	Conduct detailed field inspections of location. Document current lighting gaps and visibility issues, and gather baseline safety records and accident history.
Community Research & Data Analysis	August 11, 2026 – September 25, 2026	Distribute resident surveys, conduct interviews with commuters, and analyze the data to link lighting deficiencies with public safety concerns.
Design, Budgeting & Procurement	September 26, 2026 – November 15, 2026	Finalize technical engineering designs, determine post-spacing, calculate total project costs, and manage the sourcing and delivery of solar LED components.
Installation & System Testing	November 16, 2026 – December 31, 2026	Execute physical installation of light posts and wiring. Conduct comprehensive testing and safety inspections to ensure the system is fully operational.

Performance Review & Final Thesis	January 01, 2027 – January 20, 2027	Evaluate the real-world impact of the lighting, collect post-installation feedback, finalize the thesis manuscript, and prepare for the final presentation.
-----------------------------------	-------------------------------------	---

BUDGET

A. Material Cost

Item	Supplier	Quantity	Unit Cost	Total Cost
Solar LED Motion-Sensor Streetlight (200W, LiFeP04)	GlobeLED	20	3,800	76,000
6m Galvanized Steel Pole	Street Pole Philippines	20	8,400	168,000
Anchor Bolt / Base Plate Set	Local Fabrication	20	1,200	24,000
Concrete Foundation Materials	Yuscom Construction Supply	20	2,000	40,000
Gravel / Sand / Rebar	Yuscom Construction Supply	Lump Sum	—	35,000
Total Cost: 343,000				

B. Installation Materials

Item	Source	Total Cost
Conduits / PVC / Protection Pipes	Citi Hardware	11,000

Electrical Wiring Connectors	Royal Cord	16,000
Brackets / Clamps / Fasteners	Local Fabrication	14,000
Waterproof Sealants / Fasteners	Citi Hardware	7,000
Total Cost: 48,000		

C. Transportation Cost / Miscellaneous

Item	Total Cost
Delivery (Steel poles + Solar Units)	24,000
Permits / Barangay Coordination	7,000
Contingency (Repairs, Delays, and Replacement)	80,000
Total Cost: 111,000	

D. Installation Labor Cost

Work Description	Total Cost
Excavation, Pole Setting, Wiring, Testing (20 Units)	125,000
Total Cost: 125,000	

E. Worker Salaries (8 months, 4 days a week)

Description	Computation	Total Cost
8 Workers	600/day x 128 days x 8 workers	614,400
Total Cost: 614,400		

F. Summary of Expenses

Category	Amount
Materials	343,000
Installation Materials	48,000
Transportation/Miscellaneous	111,000
Installation Cost	125,000
Worker Salaries	614,400
GRAND TOTAL	1,241,400

References:

World Health Organization. (2018). *Global status report on road safety 2018.*

<https://www.who.int/publications/i/item/9789241565684>

Commission Internationale de l'Éclairage (CIE). (2010). *Lighting of roads for motor and pedestrian traffic* (CIE 115:2010). CIE Central Bureau.

<https://cie.co.at/publications/lighting-roads-motor-and-pedestrian-traffic>

Elvik, R. (2009). *The handbook of road safety measures* (2nd ed.). Emerald Group Publishing Limited.

https://www.researchgate.net/publication/229045126_EFFECTS_OF_ROAD_TRAFFIC_SAFETY_MEASURES_A_PRESENTATION_OF_THE_TRAFFIC_SAFETY_HANDBOOK

Painter, K. (1996). The influence of street lighting improvements on crime, fear and pedestrian street use. *Security Journal*, 7(2), 116–124.

https://www.researchgate.net/publication/359229698_Literature_study_The_effects_of_reduced_public_lighting_on_crime_fear_of_crime_and_road_safety

Welsh, B. C., & Farrington, D. P. (2008). *Effects of Improved Street Lighting on Crime*. *Campbell Systematic Reviews*, 4(1), 1–51 <https://doi.org/10.4073/csr.2008.13>

Appendix D: Relevant Source Code

D1: Base Model Configuration - contains the code responsible for defining and initializing the individual machine learning models used in the study. This includes the parameters and settings required before training the models.

```
# Cell 3: Define Model Configurations
model_configs = {
    'knn': {
        'model': KNeighborsRegressor(),
        'params': {
            'n_neighbors': [3, 5, 7, 9],
            'weights': ['uniform', 'distance']
        }
    },
    'decision_tree': {
        'model': DecisionTreeRegressor(random_state=42),
        'params': {
            'max_depth': [3, 5, 7, 10, None],
            'min_samples_split': [2, 5, 10],
            'min_samples_leaf': [1, 2, 4]
        }
    },
    'random_forest': {
        'model': RandomForestRegressor(random_state=42),
        'params': {
            'n_estimators': [50, 100, 200],
            'max_depth': [5, 10, 15, None],
            'min_samples_split': [2, 5, 10]
        }
    },
    'svr': {
        'model': SVR(),
        'params': {
            'C': [0.1, 1.0, 10.0],
            'epsilon': [0.01, 0.1, 0.2],
            'kernel': ['rbf']
        }
    },
    'ridge': {
        'model': Ridge(),
        'params': {
            'alpha': [0.1, 1.0, 10.0, 100.0]
        }
    },
    'neural_network': {
        'model': MLPRegressor(random_state=42, max_iter=1000, early_stopping=True),
        'params': {
            'hidden_layer_sizes': [(50,), (100,), (50, 50), (100, 50)],
            'activation': ['relu', 'tanh'],
            'alpha': [0.0001, 0.001, 0.01],
            'learning_rate': ['constant', 'adaptive']
        }
    }
}

print("Model Configurations:")
for name in model_configs.keys():
    print(f"    {name.upper()}")
```

D2: Base Model Training and Tracking Performance - presents the implementation for training the base models and monitoring their performance. It shows how evaluation metrics were calculated to assess the effectiveness of each model.

```
# Cell 4: Train Models with 10-Fold CV - Track During and After Training
results = {}
cv = KFold(n_splits=10, shuffle=True, random_state=42)

print("=" * 70)
print("TRAINING BASE MODELS WITH 10-FOLD CROSS-VALIDATION")
print("=" * 70)

for name, config in model_configs.items():
    print(f"\n{' '*60}")
    print(f"Training {name.upper()}...")
    print(f"{' '*60}")

    start_time = time.time()

    grid_search = GridSearchCV(
        config['model'],
        config['params'],
        cv=cv,
        scoring='neg_mean_squared_error',
        n_jobs=-1,
        return_train_score=True
    )

    grid_search.fit(X_train, y_train)
    training_time = time.time() - start_time
    best_model = grid_search.best_estimator_

    cv_results = grid_search.cv_results_
    best_index = grid_search.best_index_

    # -- DURING TRAINING: Extract per-fold train and validation scores --
    fold_train_mse = []
    fold_val_mse = []
    for fold_idx in range(10):
        train_key = f'split{fold_idx}_train_score'
        val_key = f'split{fold_idx}_test_score'
        fold_train_mse.append(-cv_results[train_key][best_index])
        fold_val_mse.append(-cv_results[val_key][best_index])

    fold_train_rmse = [np.sqrt(mse) for mse in fold_train_mse]
    fold_val_rmse = [np.sqrt(mse) for mse in fold_val_mse]

    print(f"\n [DURING TRAINING] Per-Fold Performance (Best Hyperparams):")
    print(f"   {'Fold':<6} {'Train RMSE':<14} {'Val RMSE':<14} {'Gap':<10}")
    print(f"   {'-'*44}")
    for i in range(10):
        gap = fold_val_rmse[i] - fold_train_rmse[i]
        print(f"   {i+1:<6} {fold_train_rmse[i]:<14.4f} {fold_val_rmse[i]:<14.4f} {gap:<10.4f}")

    cv_train_rmse_mean = np.mean(fold_train_rmse)
    cv_train_rmse_std = np.std(fold_train_rmse)
    cv_val_rmse_mean = np.mean(fold_val_rmse)
    cv_val_rmse_std = np.std(fold_val_rmse)

    print(f"\n During Training Summary:")
    print(f"   Avg Train RMSE: {cv_train_rmse_mean:.4f} +/- {cv_train_rmse_std:.4f}")
    print(f"   Avg Val RMSE:   {cv_val_rmse_mean:.4f} +/- {cv_val_rmse_std:.4f}")
    print(f"   Avg Fold Gap:   {cv_val_rmse_mean - cv_train_rmse_mean:.4f}")
```

```

cv_mean = np.mean(fold_val_mse)
cv_std = np.std(fold_val_mse)
cv_min = np.min(fold_val_mse)
cv_max = np.max(fold_val_mse)
cv_rmse = np.sqrt(cv_mean)

# -- AFTER TRAINING: Evaluate on training set and test set --
y_pred_train = best_model.predict(X_train)
train_mse = mean_squared_error(y_train, y_pred_train)
train_rmse = np.sqrt(train_mse)
train_mae = mean_absolute_error(y_train, y_pred_train)
train_r2 = r2_score(y_train, y_pred_train)

y_pred = best_model.predict(X_test)
test_mse = mean_squared_error(y_test, y_pred)
test_rmse = np.sqrt(test_mse)
test_mae = mean_absolute_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)

overfit_gap = train_r2 - test_r2

print(f"\n [AFTER TRAINING] Final Model Performance:")
print(f"      Training Set:")
print(f"          RMSE: {train_rmse:.4f}")
print(f"          MAE:  {train_mae:.4f}")
print(f"          R2:   {train_r2:.4f}")
print(f"      Test Set:")
print(f"          RMSE: {test_rmse:.4f}")
print(f"          MAE:  {test_mae:.4f}")
print(f"          R2:   {test_r2:.4f}")
print(f"      Overfit Gap (Train R2 - Test R2): {overfit_gap:.4f}")
if overfit_gap > 0.1:
    print(f"          WARNING: Potential overfitting detected (gap > 0.1)")
print(f"      Training Time: {training_time:.2f}s")
print(f"      Best Params: {grid_search.best_params_}")

```

D3: Ensemble Model Configuration - includes the configuration of the ensemble

learning approach, where multiple models are combined to improve prediction accuracy.

```
# Cell 3: Create Ensemble Models
# Neural network for bagging base estimator
neural_bagging_base = MLPRegressor(
    hidden_layer_sizes=(100, 50),
    activation='relu',
    alpha=0.001,
    learning_rate='adaptive',
    random_state=42,
    max_iter=1000,
    early_stopping=True,
    validation_fraction=0.1
)

# Neural network for final estimator in stacking
neural_final_estimator = MLPRegressor(
    hidden_layer_sizes=(100, 50),
    activation='relu',
    alpha=0.001,
    learning_rate='adaptive',
    random_state=42,
    max_iter=1000,
    early_stopping=True
)

# Ridge regression for final estimator in stacking
ridge_final_estimator = Ridge(
    alpha=1.0,
    random_state=42
)

ensemble_models = {
    'bagging_neural_network': BaggingRegressor(
        estimator=neural_bagging_base,
        n_estimators=10,
        max_samples=0.8,
        max_features=0.8,
        random_state=42,
        n_jobs=-1
    ),
    'bagging_random_forest': RandomForestRegressor(
        n_estimators=100,
        max_depth=10,
        min_samples_split=5,
        random_state=42,
        bootstrap=True
    ),
    'boosting_gradient_boost': GradientBoostingRegressor(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=3,
        random_state=42
    ),
    'stacking_neural_ridge_neural_final': StackingRegressor(
        estimators=[
            ('neural_network', base_models['neural_network']),
            ('ridge', base_models['ridge'])
        ],
        final_estimator=neural_final_estimator,
        cv=5,
        n_jobs=-1
    ),
}
```



```

    'stacking_neural_ridge_ridge_final': StackingRegressor(
        estimators=[
            ('neural_network', base_models['neural_network']),
            ('ridge', base_models['ridge'])
        ],
        final_estimator=ridge_final_estimator,
        cv=5,
        n_jobs=-1
    ),
    'stacking_neural_only': StackingRegressor(
        estimators=[
            ('neural_network_1', base_models['neural_network'])
        ],
        final_estimator=MLPRegressor(
            hidden_layer_sizes=(50,),
            activation='relu',
            alpha=0.001,
            learning_rate='adaptive',
            random_state=42,
            max_iter=1000,
            early_stopping=True
        ),
        cv=5,
        n_jobs=-1
    )
}

print("Ensemble Models Created:")
for name in ensemble_models.keys():
    print(f"    {name.replace('_', ' ').upper()}")

```

D4: Ensemble Model Training and Tracking - demonstrates how the ensemble model was trained and evaluated, including performance tracking to compare it with individual models.

```
# Cell 4: Train Ensemble Models with 10-Fold CV - Track During and After Training
cv = KFold(n_splits=10, shuffle=True, random_state=42)
ensemble_results = {}

print("\n" + "=" * 70)
print("TRAINING ENSEMBLE MODELS WITH 10-FOLD CROSS-VALIDATION")
print("=" * 70)

for name, model in ensemble_models.items():
    print(f"\n{'='*60}")
    print(f"[MODEL] Training {name.upper()} with 10-fold CV...")
    print(f"{'='*60}")

    start_time = time.time()

    # -- DURING TRAINING: Per-fold train and validation scores --
    fold_train_mse_list = []
    fold_val_mse_list = []

    for fold_idx, (train_idx, val_idx) in enumerate(cv.split(X_train, y_train)):
        X_fold_train, X_fold_val = X_train[train_idx], X_train[val_idx]
        y_fold_train, y_fold_val = y_train[train_idx], y_train[val_idx]

        fold_model = clone(model)
        fold_model.fit(X_fold_train, y_fold_train)

        # Training set score for this fold
        y_fold_train_pred = fold_model.predict(X_fold_train)
        fold_train_mse = mean_squared_error(y_fold_train, y_fold_train_pred)
        fold_train_mse_list.append(fold_train_mse)

        # Validation set score for this fold
        y_fold_val_pred = fold_model.predict(X_fold_val)
        fold_val_mse = mean_squared_error(y_fold_val, y_fold_val_pred)
        fold_val_mse_list.append(fold_val_mse)

    fold_train_rmse = [np.sqrt(mse) for mse in fold_train_mse_list]
    fold_val_rmse = [np.sqrt(mse) for mse in fold_val_mse_list]

    cv_train_rmse_mean = np.mean(fold_train_rmse)
    cv_train_rmse_std = np.std(fold_train_rmse)
    cv_val_rmse_mean = np.mean(fold_val_rmse)
    cv_val_rmse_std = np.std(fold_val_rmse)

    print(f"\n [DURING TRAINING] Per-Fold Performance:")
    print(f"   {'Fold':<6} {'Train RMSE':<14} {'Val RMSE':<14} {'Gap':<10}")
    print(f"   {'-'*44}")
    for i in range(10):
        gap = fold_val_rmse[i] - fold_train_rmse[i]
        print(f"   {i+1:<6} {fold_train_rmse[i]:<14.4f} {fold_val_rmse[i]:<14.4f} {gap:<10.4f}")

    print(f"\n During Training Summary:")
    print(f"   Avg Train RMSE: {cv_train_rmse_mean:.4f} +/- {cv_train_rmse_std:.4f}")
    print(f"   Avg Val RMSE:   {cv_val_rmse_mean:.4f} +/- {cv_val_rmse_std:.4f}")
    print(f"   Avg Fold Gap:   {cv_val_rmse_mean - cv_train_rmse_mean:.4f}")
```

```

cv_mean = np.mean(fold_val_mse_list)
cv_std = np.std(fold_val_mse_list)
cv_rmse = np.sqrt(cv_mean)

# -- AFTER TRAINING: Train on full training set --
model.fit(X_train, y_train)
training_time = time.time() - start_time

# Training set (variable) model: Any
y_pred_train = model.predict(X_train)
train_mse = mean_squared_error(y_train, y_pred_train)
train_rmse = np.sqrt(train_mse)
train_mae = mean_absolute_error(y_train, y_pred_train)
train_r2 = r2_score(y_train, y_pred_train)

# Test set evaluation
y_pred = model.predict(X_test)
test_mse = mean_squared_error(y_test, y_pred)
test_rmse = np.sqrt(test_mse)
test_mae = mean_absolute_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)

overfit_gap = train_r2 - test_r2

print(f"\n [AFTER TRAINING] Final Model Performance:")
print(f"     Training Set:")
print(f"         RMSE: {train_rmse:.4f}")
print(f"         MAE:  {train_mae:.4f}")
print(f"         R2:   {train_r2:.4f}")
print(f"     Test Set:")
print(f"         RMSE: {test_rmse:.4f}")
print(f"         MAE:  {test_mae:.4f}")
print(f"         R2:   {test_r2:.4f}")
print(f"     Overfit Gap (Train R2 - Test R2): {overfit_gap:.4f}")
if overfit_gap > 0.1:
    print(f"         WARNING: Potential overfitting detected (gap > 0.1)")
print(f"     Training Time: {training_time:.2f}s")

```

D5: Data Processing - provides the code used for preparing and transforming the dataset before it is used by the models. This includes data cleaning, feature preparation, and other preprocessing steps.

```
# Load data
df = pd.read_csv('../data/sdv_dataset_2.csv')
print(f"Dataset Shape: {df.shape}")
print(f"\nColumns: {list(df.columns)}")
df.head()

print("=" * 60)
print("DATA EXPLORATION")
print("=" * 60)

print("\nMissing Values:")
missing = df.isnull().sum()
if missing.sum() == 0:
    print("    No missing values!")
else:
    print(missing[missing > 0])

print(f"\nTarget Distribution:")
print(df['ExamResultPercent'].describe())

# Define column types
categorical_columns = ['Gender', 'IncomeLevel', 'EmploymentStatus']
numerical_columns = ['Age', 'StudyHours', 'SleepHours', 'Confidence', 'TestAnxiety',
                    'MockExamScore', 'GWA', 'Scholarship', 'InternshipGrade',
                    'EnglishProficiency', 'MotivationScore', 'SocialSupport']
binary_columns = ['ReviewCenter']

grade_columns = ['GWA', 'InternshipGrade', 'MockExamScore']
score_columns = [col for col in numerical_columns if col not in grade_columns]

# Prepare features and target
X_full = df[categorical_columns + numerical_columns + binary_columns].copy()
y_full = df['ExamResultPercent'].values

# Split data BEFORE any preprocessing to prevent data leakage
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_full, y_full, test_size=0.2, random_state=42, shuffle=True
)

print(f"\nTrain-Test Split Complete:")
print(f"    Training samples: {len(X_train_raw)}")
print(f"    Test samples: {len(X_test_raw)}")
print(f"    Target range (train): [{y_train.min():.2f}, {y_train.max():.2f}]")
print(f"    Target range (test): [{y_test.min():.2f}, {y_test.max():.2f}]")

# Step 1: Handle missing values - FIT ON TRAIN ONLY
print("\n[STEP 1] Handling Missing Values...")
print(f"Missing values in training data: {X_train_raw.isnull().sum()[X_train_raw.isnull().sum() > 0]}")

# Initialize imputers
iterative_imputer = IterativeImputer(random_state=42, max_iter=10, verbose=0)
median_imputer = SimpleImputer(strategy='median')

# Create copies for imputation
X_train_imputed = X_train_raw.copy()
X_test_imputed = X_test_raw.copy()

# FIT imputers on training data only, then transform both
if X_train_raw[grade_columns].isnull().any().any():
    print("\nImputing grade-based columns with IterativeImputer...")
    X_train_imputed[grade_columns] = iterative_imputer.fit_transform(X_train_raw[grade_columns])
    X_test_imputed[grade_columns] = iterative_imputer.transform(X_test_raw[grade_columns])
```

```

else:
    print("\nNo missing values in grade columns")

if X_train_raw[score_columns].isnull().any().any():
    print("\nImputing score-based columns with median...")
    X_train_imputed[score_columns] = median_imputer.fit_transform(X_train_raw[score_columns])
    X_test_imputed[score_columns] = median_imputer.transform(X_test_raw[score_columns])
else:
    print("\nNo missing values in score columns")

# Step 2: Encode categorical variables - FIT ON TRAIN ONLY
print("\n[STEP 2] Encoding Categorical Variables...")
label_encoders = {}
X_train_encoded = X_train_imputed.copy()
X_test_encoded = X_test_imputed.copy()

for col in categorical_columns:
    le = LabelEncoder()
    # Fit on training data
    X_train_encoded[col] = le.fit_transform(X_train_imputed[col].astype(str))

    # Handle potential unseen categories in test data
    def safe_transform(x):
        try:
            return le.transform([str(x)])[0]
        except ValueError:
            # If category not seen in training, use most frequent class
            return le.transform([le.classes_[0]])[0]

    X_test_encoded[col] = X_test_imputed[col].apply(safe_transform)
    label_encoders[col] = le
    print(f"    Encoded: {col} ({len(le.classes_)} classes)")

# Step 3: Scale features - FIT ON TRAIN ONLY
print("\n[STEP 3] Scaling Features...")
scaler = StandardScaler()

# Fit on training data and transform both
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)

# Final validation
assert not np.isnan(X_train_scaled).any(), "ERROR: NaN in scaled training data"
assert not np.isnan(X_test_scaled).any(), "ERROR: NaN in scaled test data"
assert not np.isinf(X_train_scaled).any(), "ERROR: Inf in scaled training data"
assert not np.isinf(X_test_scaled).any(), "ERROR: Inf in scaled test data"

print(f"\nScaling complete:")
print(f"    Training data shape: {X_train_scaled.shape}")
print(f"    Test data shape: {X_test_scaled.shape}")
print(f"    Training mean: {X_train_scaled.mean():.6f} (should be ~0)")
print(f"    Training std: {X_train_scaled.std():.6f} (should be ~1)")

```

D6: Load Selected Model Function - contains the function responsible for loading the trained model that will be used for predictions in the system.

```
31 def load_selected_model(model_name, model_category='base'):
32     """
33     Load the selected regression model and preprocessor with validation
34     """
35     try:
36         if model_category == 'base':
37             model_dir = BASE_MODELS_DIR
38             model_file = f"{model_name}_model.pkl"
39         elif model_category == 'ensemble':
40             model_dir = ENSEMBLE_MODELS_DIR
41             model_file = f"{model_name}_ensemble.pkl"
42         else:
43             return None, None, f"Invalid model category: {model_category}"
44         # Validate model file existence and integrity
45         model_path = os.path.join(model_dir, model_file)
46         # Check if model file exists
47         if not os.path.exists(model_path):
48             return None, None, f"Model file not found: {model_path}"
49
50         file_size = os.path.getsize(model_path)
51         if file_size < 100:
52             return None, None, f"Model file appears corrupted (size: {file_size} bytes)"
53         # Attempt to load the model
54         try:
55             model = joblib.load(model_path)
56             print(f"Model loaded successfully: {model_path} ({file_size:,} bytes)")
57         except Exception as e:
58             return None, None, f"Failed to load model: {str(e)}. Please re-train the model."
59
60         if not os.path.exists(PREPROCESSOR_PATH):
61             return None, None, f"Preprocessor file not found: {PREPROCESSOR_PATH}"
62         # Validate preprocessor file integrity
63         preprocessor_size = os.path.getsize(PREPROCESSOR_PATH)
64         if preprocessor_size < 100:
65             return None, None, f"Preprocessor file appears corrupted (size: {preprocessor_size} bytes)"
66         # Attempt to load the preprocessor
67         try:
68             preprocessor = joblib.load(PREPROCESSOR_PATH)
69             print(f"Preprocessor loaded successfully: {PREPROCESSOR_PATH} ({preprocessor_size:,} bytes)")
70         except Exception as e:
71             return None, None, f"Failed to load preprocessor: {str(e)}. Please re-train the model."
72
73         print(f"Successfully loaded {model_category} model: {model_name}")
74         print(f"Preprocessor details: {preprocessor}")
75
76         return model, preprocessor, None
77
78     except Exception as e:
79         error_msg = f"Error loading model: {str(e)}"
80         print(f"[ERROR] {error_msg}")
81         traceback.print_exc()
82         return None, None, error_msg
83 ..
```

D7: Prediction Function - shows how the system generates predictions based on the input data provided by the user or dataset.

```
112 def handle_prediction(request, template_name, redirect_url_name):
113     """
114     Common prediction handling logic for both student and admin views
115     """
116     if request.method == 'POST':
117         form = PredictionForm(request.POST)
118         # Validate form and prepare input data
119         if form.is_valid():
120             form_data = form.cleaned_data
121             input_data = prepare_input_data(form_data)
122
123             model, preprocessor, error = load_selected_model(
124                 model_name=SELECTED_MODEL_NAME,
125                 model_category=MODEL_CATEGORY
126             )
127             # Check for model loading errors
128             if error:
129                 messages.error(request, error)
130                 return render(request, template_name, {'form': form})
131
132             try:
133                 prediction_result = make_single_prediction(
134                     model=model,
135                     preprocessor=preprocessor,
136                     input_data=input_data
137                 )
138             # Check for prediction errors
139             if not prediction_result:
140                 messages.error(request, 'Failed to make prediction. Please try again.')
141                 return render(request, template_name, {'form': form})
142
143             predicted_class = prediction_result['prediction']
144             pass_probability = prediction_result['pass_probability']
145
146             recommendations = generate_recommendations(input_data, pass_probability)
147             risk_info = get_risk_level(pass_probability)
148
149             # Save prediction to database
150             prediction = Prediction.objects.create(
151                 user=request.user,
152                 age=input_data['Age'],
153                 gender=input_data['Gender'],
154                 gpa=input_data['GPA'],
155                 internship_grade=input_data['InternshipGrade'],
156                 study_hours=input_data['StudyHours'],
157                 sleep_hours=input_data['SleepHours'],
158                 review_center=bool(input_data['ReviewCenter']),
159                 confidence=input_data['Confidence'],
160                 test_anxiety=input_data['TestAnxiety'],
161                 mock_exam_score=input_data.get('MockExamScore'),
162                 scholarship=bool(input_data['Scholarship']),
163                 income_level=input_data['IncomeLevel'],
164                 employment_status=input_data['EmploymentStatus'],
165                 english_proficiency=input_data['EnglishProficiency'],
166                 motivation_score=input_data['MotivationScore'],
167                 social_support=input_data['SocialSupport'],
168                 prediction_result='PASS' if predicted_class == 1 else 'FAIL',
169                 probability=pass_probability * 100
170             )
171
```

```

172 # Save to prediction history
173 PredictionHistory.objects.create(
174     user=request.user,
175     age=input_data['Age'],
176     gender=input_data['Gender'],
177     study_hours=input_data['StudyHours'],
178     sleep_hours=input_data['SleepHours'],
179     review_center=bool(input_data['ReviewCenter']),
180     confidence=input_data['Confidence'],
181     test_anxiety=input_data['TestAnxiety'],
182     mock_exam_score=input_data.get('MockExamScore'),
183     gpa=input_data['GPA'],
184     scholarship=bool(input_data['Scholarship']),
185     internship_grade=input_data['InternshipGrade'],
186     income_level=input_data['IncomeLevel'],
187     employment_status=input_data['EmploymentStatus'],
188     english_proficiency=input_data['EnglishProficiency'],
189     motivation_score=input_data['MotivationScore'],
190     social_support=input_data['SocialSupport'],
191     avg_probability=pass_probability * 100,
192     risk_level=risk_info['level'],
193     base_predictions={
194         'model_name': SELECTED_MODEL_NAME,
195         'model_category': MODEL_CATEGORY,
196         'prediction': prediction_result
197     },
198     ensemble_predictions=None,
199     ip_address=request.META.get('REMOTE_ADDR'),
200     user_agent=request.META.get('HTTP_USER_AGENT')
201 )
202 # Store results in session for result view
203 request.session['prediction_results'] = {
204     'input_data': input_data,
205     'prediction': prediction_result,
206     'model_name': SELECTED_MODEL_NAME,
207     'model_category': MODEL_CATEGORY,
208     'recommendations': recommendations,
209     'avg_probability': pass_probability * 100,
210     'risk_info': risk_info,
211     'prediction_id': prediction.id
212 }
213 # Redirect to result view
214 return redirect(redirect_url_name, prediction_id=prediction.id)
215
216 except Exception as e:
217     messages.error(request, f'Prediction error: {str(e)}')
218     traceback.print_exc()
219     return render(request, template_name, {'form': form})
220 else:
221     form = PredictionForm()
222
223 return render(request, template_name, {'form': form})
...

```


D8: View Results Function - includes the code that displays or outputs the prediction results, allowing users to interpret the model's predictions effectively.

```
226 @login_required
227 def result_view(request, prediction_id):
228     try:
229         prediction = get_object_or_404(Prediction, id=prediction_id, user=request.user)
230
231         results = request.session.get('prediction_results')
232
233         if results and results.get('prediction_id') == prediction_id:
234             context = {
235                 'input_data': results['input_data'],
236                 'predictions': {
237                     'base_models': {
238                         results['model_name']: results['prediction']
239                     },
240                     'ensemble_models': {}
241                 },
242                 'model_name': results['model_name'],
243                 'model_category': results.get('model_category', MODEL_CATEGORY),
244                 'recommendations': results['recommendations'],
245                 'avg_probability': results['avg_probability'],
246                 'risk_info': results['risk_info'],
247                 'prediction': prediction
248             }
249         else:
250             # Reconstruct data from database
251             input_data = {
252                 'Age': prediction.age,
253                 'Gender': prediction.gender,
254                 'GWA': prediction.gwa,
255                 'InternshipGrade': prediction.internship_grade,
256                 'StudyHours': prediction.study_hours,
257                 'SleepHours': prediction.sleep_hours,
258                 'ReviewCenter': 1 if prediction.review_center else 0,
259                 'Confidence': prediction.confidence,
260                 'TestAnxiety': prediction.test_anxiety,
261                 'MockExamScore': prediction.mock_exam_score,
262                 'Scholarship': 1 if prediction.scholarship else 0,
263                 'IncomeLevel': prediction.income_level,
264                 'EmploymentStatus': prediction.employment_status,
265                 'EnglishProficiency': prediction.english_proficiency,
266                 'MotivationScore': prediction.motivation_score,
267                 'SocialSupport': prediction.social_support
268             }
269
270             prediction_result = {
271                 'prediction': 1 if prediction.prediction_result == 'PASS' else 0,
272                 'pass_probability': prediction.probability / 100,
273                 'predicted_score': prediction.probability,
274                 'prediction_label': prediction.prediction_result
275             }
276
277             recommendations = generate_recommendations(input_data, prediction.probability / 100)
278             risk_info = get_risk_level(prediction.probability / 100)
279
```

```

280         context = {
281             'input_data': input_data,
282             'predictions': {
283                 'base_models': {
284                     SELECTED_MODEL_NAME: prediction_result
285                 },
286                 'ensemble_models': {}
287             },
288             'model_name': SELECTED_MODEL_NAME,
289             'model_category': MODEL_CATEGORY,
290             'recommendations': recommendations,
291             'avg_probability': prediction.probability,
292             'risk_info': risk_info,
293             'prediction': prediction
294         }
295
296     # Determine template based on user role
297     if request.user.role == 'student':
298         template_name = 'pages/student_result.html'
299     else:
300         template_name = 'pages/admin_result.html'
301
302     return render(request, template_name, context)
303
304 except Prediction.DoesNotExist:
305     messages.error(request, 'Prediction not found.')
306     return redirect('prediction:history')
307

```
